

Dynamic Environments affecting Player Behaviour and Perception compared to Static Platforms

Venkata Sanjay Varma Penmatsa

Msc Computer Games Technology, 2026.

School of Design and Informatics
Abertay University

Table of Contents

Table of Figures	ii
Table of Tables	iii
Acknowledgements	iv
Abstract.....	v
Chapter 1 Introduction	1
Chapter 2 Literature Review	4
2.1 Platformer Traversal and Level Design	4
2.2 Environmental Affordances in Games	6
2.3 Dynamic Systems, Emergence and Player Agency	9
2.4 Evaluating Player Traversal Behaviour	11
2.5 Evaluating Player Perception	13
2.6 Summary and research gap	14
Chapter 3 Methodology.....	17
3.1 Game Prototype	18
3.2 Experimentation	29
3.3 Data Analysis.....	33
Chapter 4 Results and Discussion	36
4.1 Overall Traversal Performance	36
4.2 Segment-Level Traversal Differences	38
4.3 Deaths and Traversal Failure	39
4.4 Dynamic Cloud Interaction	41
4.5 Traversal Routes and Spatial Behaviour.....	42
4.6 Player Perception	44
4.7 Overall Findings	46
Chapter 5 Conclusion and Future Work.....	48
List of References	51
Bibliography	55
Appendices	56

Table of Figures

Figure 3.1: Main control instructions shown before both game levels	19
Figure 3.2: Dynamic-condition instructions explaining aiming, bounciness and moveable cloud states	20
Figure 3.3: Schematic top-down representation of the static level	21
Figure 3.4: Top-down view of the three routes in the third segment of the static level	23
Figure 3.5: Visual representation of the dynamic cloud system	24
Figure 3.6: C++ Function used to reposition the moveable cloud	25
Figure 3.7: State breakdown of the breaking cloud	27
Figure 3.8: Schematic top-down representation of the dynamic level	28
Figure 3.9: Class diagram for Gameplay Telemetry	30
Figure 3.10: Algorithm to visualise traversal paths	34
Figure 3.11: Algorithm to visualise death locations	35
Figure 4.1: Static and Dynamic completion times by participants	37
Figure 4.2: Static and Dynamic idle duration by participant	38
Figure 4.3: Death locations recorded in the static level	40
Figure 4.4: Death locations recorded in the dynamic level	40
Figure 4.5: Dynamic cloud interaction by participant and interaction type	41
Figure 4.6: Simplified traversal paths from the players in the static level	42
Figure 4.7: Simplified traversal paths from the players in the dynamic level	43
Figure 4.8: Table representing the participant questionnaire responses	44

Table of Tables

Table 3.1: Dynamic cloud states	17
Table 4.1: Summary of gameplay telemetry for static and dynamic levels	36
Table 4.2: Segment data for static and dynamic levels	39

Acknowledgements

I would like to thank my supervisors for their guidance, feedback and support throughout the development of this dissertation. Their advice has been very valuable in creating the research prototype, completing the analysis and final presentation of the study. I would also like to thank all of the participants who took part in the playtest sessions and gave their time and feedback for this research.

Abstract

Platformer games traditionally require the players to navigate the environment where the platform positions and behaviours are mainly predetermined by the level designer. This dissertation explores the idea of letting the players deliberately change the states of the environment to affect the traversal behaviour and player perception in a 3D platforming environment.

A 3D platforming prototype was developed in Unreal Engine 5 containing two conditions. The Static condition used standard pre-determined platform behaviours and the Dynamic condition gave players the opportunity to manipulate platform behaviours through various environmental states. Ten participants completed both the conditions. Gameplay data like completion time, total jumps, deaths, idle behaviour, traversal paths and environmental interactions were collected and a post play questionnaire was used to examine perceived difficulty, environmental interactions, route freedom and enjoyment.

The outcomes showed that there were some differences between the two conditions. For most participants, the Dynamic level took longer to accomplish and resulted in a large increase in the number of idles recorded, but the number of deaths were similar. The Dynamic condition was also found to have more differences between participant traversal paths in spatial visualisations. The overall results were favourable to dynamic condition since most players reported greater route freedom and more enjoyment, although the dynamic level was also considered more difficult.

These results suggest that player-interactable environmental state changes might affect the pacing of traversal, route choice and player-perception. However, the small sample size, fixed play order and differences between the level layouts limit the strength of causal conclusions. The study therefore provides an exploratory foundation for further research into player-modifiable environments as a traversal design approach in 3D platforming games.

Chapter 1 Introduction

Platforming games are designed around the idea of traversal, meaning the player needs to navigate through an environment by running, jumping and passing over platforms and obstacles. The design of these environments has an important influence on how players progress through a level. Platform positioning, spacing, action and movement can dictate the routes and actions required by the player. Similar studies into platformer level design have also identified platforms, obstacles and movement elements as key features that contribute to the nature of traversal (Smith, Cha and Whitehead, 2008).

Games such as *Donkey Kong* helped establish jumping and vertical movement as central mechanics, while later titles such as *Super Mario Bros.* developed these ideas into more structured side-scrolling levels. The move into 3D, seen in games such as *Super Mario 64*, expanded platforming by allowing players to navigate environments with greater freedom across multiple directions rather than following mainly linear routes. Despite changes in technology and level complexity, traversal has remained at the centre of the genre, with much of the player experience depending on how movement abilities interact with the design of the environment. This makes platformers particularly suitable for investigating how changes to environmental behaviour can influence the way players navigate and make traversal decisions.

In most platformers, the platform functionalities are set by the level designer. Usually, the platform does not move on its own, it can be stationary (still) or move along a fixed path or break when the player touches it, but the player reacts to the actions of the platform but does not interact or change the environment around them to improve their experience. This leads to route choice being largely determined by the choice of traversal opportunities that have been already prepared in level design.

Giving players a way to alter the environment creates a different kind of traversal. Rather than simply choosing between the platforms on which to play, the player can also alter the environment or its actions prior to playing. This can impact on how players pick their routes, how quickly they run them, and how free and in control they feel in a level. Research to date has looked at platform design, player movement, and environmental affordances in platformers, but fewer works have

directly compared the static platformer with dynamic platforming level where traversal can be achieved by interacting with the environment and changing its dynamic states.

This dissertation explores this problem with the use of a 3D platforming prototype with two gameplay conditions. The first type is based on the traditional platform behaviour, where the traversal environment is designed by the level designer and cannot be altered. The second adds a dynamic environment system which when player interacts will change the functionality of the platforms in the level. The study is comparing not to see which of the approaches gives a better level, but to investigate how the player's behaviour changes when they were given greater control over the environment.

The research question guiding the study is:

How do player-driven environmental state changes affect player traversal behaviour and perception in a 3D platforming game compared with static platform behaviour?

To achieve this aim, the study has the following objectives:

- To design and develop two comparable 3D platforming levels representing static and dynamic environmental behaviours.
- To record and compare player traversal behaviour across the two conditions using gameplay telemetry.
- To examine differences in traversal patterns, completion behaviour and interaction with environmental mechanics.
- To evaluate how players perceive difficulty, environmental interaction, route freedom, navigation and enjoyment across the two conditions.

The goal of this study is to empirically examine the contribution of player controlled environmental state changes as a traversal design approach. The project aims not to simply manipulate the environment for the player to navigate, but to see how giving the player the power to change the traversable parts of the environment can change their movement and their perception of the level. This study contributes a comparative behavioural analysis of static and dynamic

platforming environments, a telemetry-based methodology for measuring traversal patterns, and design insights for player-driven level design. The results can be helpful to designers looking into more interactive or player-driven platformer level design.

The rest of the dissertation follows the outline. The Literature Review looks at past studies that have been conducted on the topic of platformer traversal, environmental affordances, dynamic systems, player agency and how to measure player behaviour and perception. The Methodology outlines the game prototype, experimental setup, and information gained from the game-play telemetry and questionnaire in the experiment. The Results and Discussion chapter reports and discusses the results of both data sources. The Conclusion and Future Work section concludes the findings of the main results, the limitations of the study and potential directions for future research are discussed.

Chapter 2 Literature Review

2.1 Platformer Traversal and Level Design

Platformer level design is fundamentally about arranging space around what the player is able to do while moving. Platforms, gaps, hazards and movement aids do not function as isolated objects - together, they determine where a player can go, what actions are needed and in what order. A platforming level can be thought of as a series of spatial problems through which the player learns and applies movement mechanics. Research in this area consistently treats level geometry and player movement as two sides of the same coin: the dimensions and behaviour of the environment set out what's possible, while the character's movement system determines which of those arrangements are actually traversable (Smith, Cha and Whitehead, 2008; Compton and Mateas, 2006).

Player movement is crucial in platformers, as movement is typically designed around repeated motions like running, jumping and controlling the character while it is in flight. These actions dictate the distance and/or height that a player can reach and hence the practical limits of a level design. The ability to jump was used as a central mechanic when analyzing platformer levels and it was demonstrated that the probability of completing a level is closely related to the difficulty of each jump action (Aramini, Lanzi and Loiacono, 2018). Movement is also strongly related to what is often described as game feel. Additionally, tuning the physical behaviour and responsiveness of game objects can help with predictability and understanding of the consequences of actions (Pichlmair and Johansen, 2022). Movement mechanics and the geometry of the level cannot be separated: altering the distance of jumping, the speed of movement, or the aerial control, for example, can alter which platforms are able to be reached and how hard a route might be.

The Platformer framework is not only a classification of platforms as objects, but also as elements of larger areas of challenge. A single platform could be a landing spot, but its gameplay role will depend on its location with respect to neighboring platforms, obstacles and abilities (Smith, Cha and Whitehead, 2008). Depending upon its dimensions, elevation and location, the same platform may serve as a simple landing point, as a temporary holding area, or form a component of a much more demanding series. It's not possible to accurately assess platform difficulty by individual objects alone. It is the result of the geometric relationship between the environment and what it needs to be done in it. The framework was developed

based on 2D platformers, and therefore does not consider other aspects such as camera control and depth movement which may be present in 3D platformers.

Gap width decides whether a normal jump, sprint jump or double jump is needed, while the size of the destination platform sets how precise the landing has to be (Wehbe et al., 2017). Height differences can force the player to pick a particular movement ability, and moving platforms add a timing element by making the player predict when a surface will actually be reachable (Aramini, Lanzi and Loiacono, 2018). These sorts of factors tend to overlap rather than act separately like a jump toward a small moving platform might demand distance judgement, timing and mid-air adjustment all at once, which produces a very different kind of challenge from a longer jump toward a large stationary surface.

Perspective is especially relevant for 3D platformers. In a side-scrolling game, forward movement and jump distance are usually laid out across a single, consistent 2D plane. In 3D, the player also has to read depth, manage the camera and judge landing positions that might be offset both vertically and laterally (Naftis, Tsatiris and Karpouzis, 2021). A jump can end up feeling difficult even when it's mechanically doable, simply because the player has incomplete or misleading visual information about where the target actually is. So, level design has to account for visibility, camera orientation and how readable a landing surface is, on top of whatever the movement system can technically achieve.

In addition, the difficulty will depend on the pacing of challenges over time. Platformer levels are designed with some key elements in mind such as repetition, rhythm and connectivity (Compton and Mateas, 2006). Individual challenges are connected to one another through connectivity, and repetition and rhythm give it form as movement through the route. Replaying a type of jump allows the player to practise it, variation ensures that it is not predictable and moments of less pressure allow the player to look forward before moving onto the next challenge. Thus, pacing is only a small part of the difficulty, but also requires actions to be added, repeated and played in a controlled order rather than all at once.

Level structure is used to introduce mechanics through gameplay instead of relying on explicit instructions (Cerny Green et al., 2018). A player might first jump between two large stationary platforms, then make the same jump across a wider gap, then later combine it with sprinting or a moving target. The challenge does

not come from constantly introducing brand-new inputs but instead it comes from changing the context those familiar actions have to be performed in

Recurring design patterns help structure these sequences too (Dahlskog and Togelius, 2012). A gap pattern may be wide or narrow and may contain hazards or contain multiple possible landing options, and a multiple-path pattern may provide different levels of risk or efficiency, but still provide the same ultimate objective. The usefulness of these patterns is that they structure the players' decisions when the geometry compels the players to continue moving forward, to slow down and observe, or consider other routes.

This does not mean having several visible platforms on screen automatically creates meaningful route choice. A route can exist geometrically and still be functionally pointless if it's slower, harder and offers no real payoff and the opposite problem shows up when one route is so obviously better that everyone just takes it. Multi-route design that actually works needs differences players can recognise and weigh up, like a shorter but more precise route against a longer one with safer landings. Those routes also need to stay tied to shared progression points, so that variation in how players move does not leave them lost about where the level is actually heading.

Platformer design research provides a useful foundation for traversal and how it is influenced by geometry, timing and rhythm, and route structure. Most platform properties are still used in games such as *Celeste* and *Super Mario 64*, where platforms are typically set up by the game designer and the player is instructed to act on them rather than manipulate them.

2.2 Environmental Affordances in Games

Recognising a game environment isn't only about identifying what objects are in the game, it's also about recognising what they allow the player to do. The form, position and actions of a platform can imply a great deal, such as: a platform can be stepped upon, jumped off, stepped over, pushed across or used to get to another location. Typically, these possibilities for action are discussed in terms of affordances, and in the design of platformers, affordances can help to explain how

traversal spaces are read by the player and what aspects of the environment are actually pertinent to them.

According to Gibson's ecological theory of perception, affordance is not just a property of an object (Gibson, 1979), but rather an action possibility that occurs with the relationship between a person and the environment. Depending on the size of the character, their movement abilities and location on the surface, a surface may provide standing space for one player, but not another. A platformer could have a platform that a character with double jump could get across, but the character with a normal jump couldn't. The environment in this case hasn't changed, but the ability of the player-controlled character has.

Affordance theory is tied so closely to platformer level design (Cardona-Rivera and Young, 2014). Platform spacing, height and motion determine whether a route is physically possible, but traversal also depends on whether the player actually notices that possibility. This distinction is important since a route can be entirely valid physically and still go unused if the player cannot identify it as a usable surface, does not understand how a platform behaves, or does not realise a particular ability is required. So, traversal depends both on what the environment technically allows and on what the player actually believes is possible to do.

Signifiers are used as the visible or perceivable cues that show where and how an interaction can happen (Norman, 2013). In games, colour, shape, animation, sound and visual effects can all do this job like a cracked platform suggests it's about to break, a surface repeatedly moving along a fixed path signals that it can carry the player, and a glowing object hints that it's interactive.

That distinction matters because a game's mechanics are not always readable just from how something looks. A platform does not naturally tell you whether it's solid, bouncy, movable or temporary but its function comes from the game's underlying rules. This means it has to be taught through consistent feedback. If a platform changes shape, colour or animation once it becomes bouncy, that change is what signals its new behaviour. Without that kind of feedback, difficulty ends up coming from unclear communication rather than from the traversal challenge the level was actually designed to test.

Learning within a game can be seen as improving perception and performance based on affordances provided by the environment (Linderöth, 2010). Hence,

development may not necessarily involve the acquisition of a new action but it may also imply discovering a solution which existed in the environment before but was not recognized instantly. For example, a player can initially see a platform only as a surface where one can land and subsequently discover that this platform can also be used to reach higher altitude or can be moved to a different position. In other words, the more knowledgeable players become about the game, the more solutions the same environment allows them to perceive.

That process comes down to a balance between instruction and exploration. Explain every possible use directly, and players understand the mechanic but have little reason to experiment. Give too little guidance, and they miss essential interactions and just find the level confusing. Good affordance design needs to communicate an object's basic function while still leaving room for players to find their own applications for it.

Platformer levels can be represented through paths and affordances, with functions like solid, moving, passable, breakable and climbable surfaces (Sarkar et al., 2020). That's a useful approach because it describes level elements by what actions they support rather than how they look. A moving platform and a breakable platform might sit in similar positions, but they create very different timing demands, risks and traversal possibilities.

However, this representation primarily describes affordances that are already associated with elements in the level. A moving surface supports one form of traversal, while another surface may provide a different action possibility. This leaves a further question for player-modifiable environments: what happens when the player can deliberately alter which affordance the same environmental element provides during play? In such a system, the player is not only recognising and selecting between existing traversal opportunities, but can also influence which opportunities become available. So, the player is not just choosing between existing affordances but they're activating different ones.

If the level geometry makes one state obviously necessary in a given spot, the interaction risks becoming more of a disguised key-and-lock puzzle than a real decision. For the system to genuinely support varied traversal, more than one state or route needs to stay practically useful, and the consequences of choosing between them need to be understandable to the player. Affordance theory gives

this study a solid foundation because it connects environmental design, player capability and perception. It explains why a mechanically available action can go unused if it is not clearly communicated, and why the same object can support different actions depending on its state.

2.3 Dynamic Systems, Emergence and Player Agency

Dynamic game systems are systems in which the game environment varies depending on the actions of the player, but does not stay fixed to their initial state. A static platformer level is one in which the player has to react mostly to the conditions set by the designer. Platforms can traverse fixed paths and may be activated by an event that causes a specific consequence, and the path taken in the level is largely predetermined prior to the start of play. A dynamic system does not do that, because it allows the player to modify a portion of the environment and affect what can be accomplished next.

The Mechanics-Dynamics-Aesthetics framework helps explain this relationship. Mechanics are the rules and systems created by the designer, dynamics are the behaviour that emerges when players interact with those systems, and aesthetics is the experience of the player during the session (Hunicke, LeBlanc and Zubek, 2004). Mechanics are logic created by the designer that performs an assigned task. Examples of dynamics are the different gameplay experiences that come out when the players interact with the logic. The resulting feelings of control, freedom, challenge or enjoyment create part of the player experience.

This difference is useful because it separates what the designer directly creates from what actually happens during a play. A designer may choose a platform to be bouncy when it is selected, but cannot fully dictate how that platform will be used by each player. One player can go onto the platform to access a higher platform while the other player can move the platform to make it easier and safer for another player to get up on or another different player can ignore the platform until he/she fails somewhere else. The same mechanic can result in different endings, dependent on the players choices, movement and level comprehension.

Emergence is commonly used in game design to describe more complex gameplay behaviours that develop from the interaction of comparatively simple rules. The relationships between game mechanics can produce emergent

gameplay as a system develops over time (Dormans, 2011). However, the term should be used cautiously. Game studies have sometimes treated openness, variation and emergence as if they were the same concept, even though the existence of several possible outcomes does not necessarily mean that genuinely emergent behaviour has occurred (Soler-Adillon, 2019). For the purposes of platformer traversal, the important point is therefore not that every alternative route should be labelled emergent, but that interacting mechanics can create a wider space of possible player strategies than a completely predetermined sequence.

Emergent traversal does not mean that the designer gave up control over the level. The designer chooses the location of platforms, the type of interactions that can be used, the character's ability, and where the player will ultimately have to travel. But that's not to say the path between those points isn't fully determined; it is, but the player still has some flexibility in the way the mechanics are used together.

Progression-based games typically present a series of challenges arranged by the designer, while emergent games depend more on rules that can create different outcomes through interaction (Juul, 2005). Adams and Dormans (2012) similarly discuss both emergence and progression as structures that can be combined within game-mechanic design. This combination is particularly relevant to level-based games, where overall progression can remain designer-controlled while local gameplay still allows different strategies or outcomes. Games such as *Portal 2* show how environmental systems can support different forms of problem solving. Portals change how spaces connect and allow ordinary surfaces to gain new traversal functions. *The Legend of Zelda: Tears of the Kingdom* similarly gives players tools to move and combine environmental objects, often allowing several solutions to the same traversal problem. These games are considerably more complex than the system used in this study, but they show how environmental interaction can shift traversal from simply following a route to partly creating one.

The presence of several options does not automatically create meaningful agency. Agency involves the player being able to make choices that produce clear outcomes. Agency also depends on the relationship between player intention, actions available to them and visible results (Wardrip-Fruin et al., 2009). A game may offer several choices, but those choices carry little weight if they all lead to

the same outcome or if the player cannot understand what difference they actually make. Meaningful play is a clear relationship between player action and the final outcome, an idea especially relevant to dynamic environments, since state changes need to produce understandable results (Salen and Zimmerman, 2004).

Difficulty can also vary according to dynamic systems. Does it become easier to play the game if players can interact with the environment? The research that has already been done previously on agency does not imply that more choice will necessarily make an experience easier or more fun.

The literature on dynamic systems, emergence and agency provides a useful basis for analysing the dynamic platforms used in this project. It shows how simple state changes can influence later actions, how different strategies can develop from the same underlying rules, and why visible choice is not always the same as meaningful agency.

2.4 Evaluating Player Traversal Behaviour

Just seeing if the player makes it to the end of the level is not enough to analyse the player's traversal behaviour. Completion time can refer to performance as a whole but it does not indicate any route taken, or any difficulty that was encountered, or how the player interacted with the mechanics provided. During game play, the interaction between players and the game can be recorded, and can thus be used to augment more traditional user testing methods (Drachen, Canossa and Sørensen, 2013).

A gameplay metric value is dependent on the behaviour being studied. The completion time can be used to represent performance, and the spatial positions, interaction events and failure locations can be used to represent the way in which the performance unfolded. However, telemetry is not about what a player wants to do, it's about what a player does. Recorded actions may not explain why a player went with a particular action, but may show what the player did. This is particularly true when comparing traversal paths as it may be the same number but different routes/strategies.

When the research is about traversal, spatial data is very relevant. Player-position data can also be analysed to determine how users navigate game environments.

A path graph may show if different players take the same path, or if they venture to the edges of the level or revisit previous areas that they repeatedly failed. This is helpful for the present study because the dynamic level is designed to support different ways of moving between shared checkpoints. Visualising movement paths can make these differences easier to identify than using statistical results alone. Visual representations of gameplay data can help researchers detect patterns that may not be clear in tables of raw values (Wallner and Kriglstein, 2013). For example, two players may have similar completion times but take very different routes. A player might assume one path easier while another player might think the opposite and decide to go on a different path.

Another useful metric is the death locations. Drachen and Canossa (2009) demonstrate this through spatial visualisations of player deaths, where areas with high concentrations of deaths can be identified and compared with different causes of failure. Such visualisations can identify sections of a level where players repeatedly encounter difficulty. However, a concentration of deaths does not by itself explain the cause of that difficulty. Repeated failures may relate to level geometry, player execution, misunderstanding of a mechanic or differences in individual play style. Death-location data should therefore be treated as evidence of where failures occur rather than direct evidence of why they occur.

Temporal measures also require cautious interpretation. If a player is idle for a few seconds, this may indicate hesitation, observation, waiting or decision-making, but the recorded pause alone cannot distinguish between these behaviours. More generally, telemetry events gain meaning from the context and sequence in which they occur rather than from frequency alone. The Play-Graph approach illustrates this broader principle by representing gameplay as sequences of states and player actions rather than treating individual events in isolation (Wallner, 2013). This suggests that temporal or interaction measures should be interpreted alongside other evidence instead of mistaking them as a single source of evidence.

Previous studies on gameplay analytics, therefore, confirm that using multiple behavioural measures is beneficial. Aggregate metrics can provide information about overall performance, spatial telemetry can reveal movement and failure patterns, and event-based data can show how interaction develops during play. These patterns can be further identified and compared visually (Wallner and Kriglstein, 2013; Wallner, 2013). But there is no single telemetry measure that

accounts for all player behaviour. It does not give any indication why a player was slower if the completion time was longer, it does not explain any failure if they died, it does not explain the intention of the player if they moved around. Behavioural telemetry is thus a good indicator of events during the gameplay and supplemental evidence is needed when the research question also includes how the events were interpreted/experienced by the player. The next section examines the use of player-perception measures to give that other vantage.

2.5 Evaluating Player Perception

Gameplay data can indicate what the player has done, but not necessarily how or why the player acted a certain way. If the completion time is longer, this could be due to confusion, careful preparation, exploration or problems with controls. Player perception needs therefore to be analysed in conjunction with traversal data to gain insight into how the player perceived the two-level conditions.

Games research often uses questionnaires because they enable the players to report what they have experienced that cannot be directly recorded from a game. These include pleasure, difficulty, control, frustration, immersion and comprehension. The Game Experience Questionnaire is designed to assess various components of the player's experience, such as competence, immersion, flow, tension, challenge, positive and negative affect (IJsselstein, de Kort and Poels, 2013). They illustrate that the level of experience is not a single evaluation and a level can be hard to master and fun to play, or easy to master but unappealing.

The Game Flow model also connects enjoyment with factors such as clear goals, feedback, player control, suitable challenge and concentration (Sweetser and Wyeth, 2005). Game Flow puts more focus into game design factors of clear goals, feedback and player control. These are things that come into play in dynamic traversal, the fact that players have choices in the environment does not make it a better experience if they do not know what they can do, or if it too hard to understand how the mechanics function.

Closed and open questions ask different types of information on the players' experience. A closed rating scale question gives participants the least freedom to interpret the question, while an open-ended question gives participants the most

freedom to interpret the question. Brühlmann and Mekler (2018) state that question type is crucial in the design of a survey when they talk about Games User Research and they note that the design of a survey can impact the quality of the data collected. A combination of structured responses and written explanations can then give comparable measures of players' perceptions and add to the interpretation of these measures.

Self-reported data has its own limitations. At times, the participants may miss some part of the session, may misunderstand the question, or may answer the question they think is expected of them (Brühlmann and Mekler, 2018). Players can claim greater route freedom even if their path is very similar to that of other players. Conversely, when the players do not, they might make different choices without being aware of it. The results from the questionnaire should therefore not be interpreted as a direct proof of behaviour.

Differences between behavioural and self-reported results may also be meaningful in themselves. A participant may take longer in the dynamic condition but still prefer it because the cloud system provides more control and opportunities to plan. Another participant may complete it quickly but report that the correct state was too obvious, suggesting that the available choice was limited. Rather than expecting every measure to produce the same result, the analysis should consider how the findings explain different parts of the experience.

Player perception is therefore necessary for determining whether environmental state changes were understood as meaningful traversal tools rather than additional obstacles. Telemetry can reveal how the clouds were used, while questionnaire responses can show whether players felt that those interactions affected their choices. Bringing these forms of evidence together provides a stronger basis for answering the research question than either method could provide alone.

2.6 Summary and research gap

The literature reviewed in this chapter shows that platformer traversal is shaped by the relationship between player movement, level geometry and the organisation of challenges. Platform spacing, movement behaviour and route structure can impact on what traversal options are available, and affordance

theory explains why some are not recognised and understood by the player. The traversal aspects of existing platformer frameworks are typically described by affordances or properties of the designed level structures (Smith, Cha and Whitehead, 2008; Sarkar et al., 2020). Player-modifiable environments have a very different relationship, as interaction may alter the nature of what an environmental element allows the player to do during play.

Dynamic systems research, emergence and agency offer a more general understanding of how players' interaction can lead to various behaviours given a common set of rules. Just because there are multiple actions and outcomes, though, does not automatically mean there is evidence of emergence or meaningful player agency. Emergence is not just the presence of several options and agency requires a connection between the player's intention, what the system does, and what the player observes. This distinction is significant because several environmental states do not necessarily mean that players feel more in control when they were in the state or use the state to create meaningfully different routes for traversal if the player can modify them.

Across the literature reviewed in this chapter, platformer research provides detailed accounts of player movement, designed level structures, difficulty, recurring patterns and environmental affordances. Research on dynamic systems and agency separately provides theories for understanding how rules can support different behaviours and how players can experience meaningful control. However, these areas are generally considered from different perspectives. The focus of platformer research is on the opportunities for traversal provided by the level design, while the focus of research on emergence and agency is on player choice and system behaviour at a macro level. This leaves a more specific gap regarding changes to the state of the environment by the player during traversal in the platformer setting, and how these changes influence the observable player traversal behaviour and the player's perception of the effect of the changes on the state of the set.

This study aims to fill this gap by examining the differences between the behaviour of a static platform level and a dynamic platform level where players can intentionally alter states of the environment as they traverse. It explores player behaviour and player perception to investigate how environmental modification alters the way in which players navigate through the level and how they perceive interaction, route freedom and challenges. The study is designed to be more

focused by combining platformer traversal, environmental affordances, and player agency in the same controlled comparison to gain a more focused understanding of player-modifiable environments as a form of 3D platformer traversal design.

Chapter 3 Methodology

The project used a within-subject comparison to investigate how player-interactable environment changes affect player behaviour and perception in platformers. Each participant is instructed to play two versions of a platforming level. One is a static level where the platforms are pre-coded and placed according to the design made by the designer while the second version is a dynamic level where multiple clouds are provided to the player and also given the ability to manipulate these platforms to complete the level. Using the same participants across different versions showed differences in traversal behaviour.

Both levels were organised in 3 segments which are separated by multiple checkpoints. This provided a common progression structure that is consistent across both the levels which helps in comparing the overall performance as well as behaviour. Having three segments lets us go in depth and find some particular differences in a smaller region of the level. The movement mechanics are the same in both static and dynamic levels. The platforms in both levels are in the shape of clouds so the paper will be addressing them as cloud platforms in the methodology. The dynamic states are static, bouncy, moveable and breakable.

Cloud State	Dynamic behaviour
Static	Basic rigid platform.
Bouncy	Launches the player in the z axis.
Moveable	Can be repositioned in the XY plane.
Breakable	Will break into small fragments upon contact and joins back after a short duration.

Table 3.1: Dynamic cloud states

A mixed-method approach was taken to examine both behaviour and perception of the players. Gameplay telemetry was automatically recorded using log files which stored the player positions, deaths, jump count, idle events and more such data. After playing both the levels, the players were asked to fill a questionnaire

which helped figure out the player traversal choices and their understanding of the levels and how they completed them.

Combining these two data allowed the study to compare what the participants actually did during the traversal with how they described their experience afterwards. This is important to address the long completion times and increased idle periods. Since it is hard to know if the players were confused with the level or just planning on how to proceed. Therefore, Gameplay telemetry and questionnaire together will provide differences between the static and dynamic conditions.

3.1 Game Prototype

3.1.1 Player Controls and Interaction

The player character uses the same main movement system in both the static and dynamic levels so that differences in player behaviour could be examined without also changing the character's basic traversal logic. Unreal Engine 5 was the game engine used for the prototype. Enhanced Input system was used to separate the main gameplay actions into individual input mappings. Player controls include movement, camera control, jumping, sprinting, aiming, cloud activation and hotbar selection. By having separate input actions, the control scheme became easier to manage across both the conditions. There was a logic in place to only enable the cloud interactions and hotbar in the dynamic level.

Movement was controlled using the WASD keys and also arrow keys for accessibility, while the mouse controlled the camera rotation. The character can also walk, sprint, jump and double jump in both conditions. A normal jump was sufficient to cover short gaps while sprint jump was used to reach greater horizontal distance. Double jump gave the player more air control and freedom to land where the player chooses to. Giving the player more than one traversal action allowed the levels to present more movement specific challenges without having to introduce new controls during the play session.

Before either level started, players were shown the control scheme ui explaining all the traversal controls available for them. This screen also provided a small tip about using sprint + jump together. By providing these instructions at the start, All

the participants began with the same basic knowledge about the control system rather than relying on experimenting with the controls. This was important because the study aimed to compare traversal decisions and not how quickly the players discover different movement techniques.

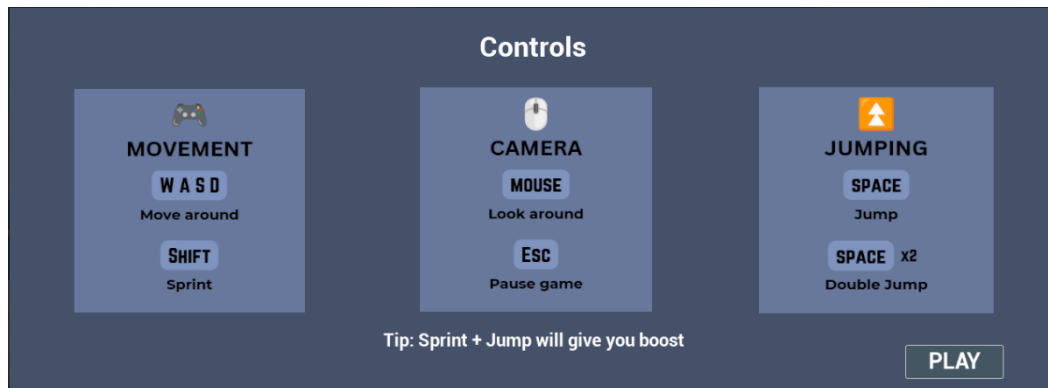


Figure 3.1: Main control instructions shown before both game levels.

The dynamic level introduced an additional interaction layer to manipulate the cloud platforms. Players can enter an aiming state when holding the right mouse button and use the left mouse button to activate the selected cloud behaviour. A ray was projected from the centre of the screen into the game world, and the cloud interaction was applied if the ray hit an object that implements the dynamic cloud interface. The study wanted to examine player driven environmental changes rather than passive environmental changes.

These cloud states can be used with the help of a small hotbar. This hotbar contains three keys to switch the state of a cloud to either bouncy, moveable or static. This was simple and reduced unnecessary control complexity while still making the player pick the environment interaction they wish. Before entering the dynamic level, players were also shown an instruction screen explaining the aiming system and how some of the states work. These instructions explained the function of the mechanics but also did not give a complete solution on how to make a path to victory.

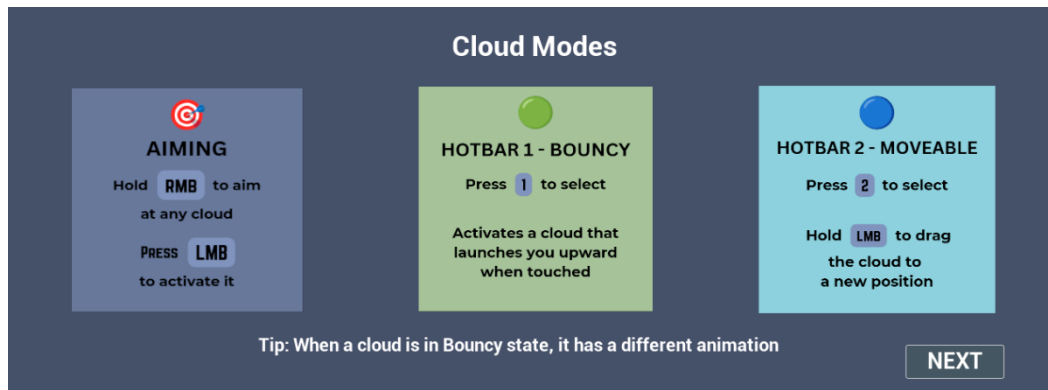


Figure 3.2: Dynamic-condition instructions explaining aiming, bouncy and moveable cloud states.

3.1.2 Static level

The static level was designed as the conventional comparison level for the study. The main purpose of it is to provide a platformer environment in which the player's movement and mastery matter the most and also the behaviour and position of platforms were predetermined by the level design. This created a baseline that can be used to compare with the dynamic level, where the players were given control over the platform mechanics. The overall structure of the static level was mainly influenced by the approach described by Smith, Cha and Whitehead (2008). The main idea is to introduce fundamental platforming mechanics first before combining them into a much more demanding traversal challenge. This was implemented so that the players could first become familiar with the available movement logic before having to master it.

The static level was divided into three main segments which are separated by checkpoints with the final goal placed at the end of the third section. This structure was implemented because it provided a clear progression in difficulty across the whole level and secondly, it provided an in-depth telemetry with the completion times and death positions of each segment. This made it possible to identify whether the differences between the static and dynamic levels were concentrated at specific segments rather than being evenly distributed across the whole level.

Checkpoints were placed throughout the level so the players can respawn at the most recently passed point if they die. There are multiple checkpoints in each segment since the intention of this study is not to punish players for dying a lot. If

a player returns all the way to the start just because they mis-timed a jump, it would be demoralizing and also provide incorrect completion times. This checkpoint system therefore reduced unnecessary repetition while still having consequences for failing in each segment. A schematic top-down view of the level was used to visualise the overall progression, stationary platforms and moving platforms.

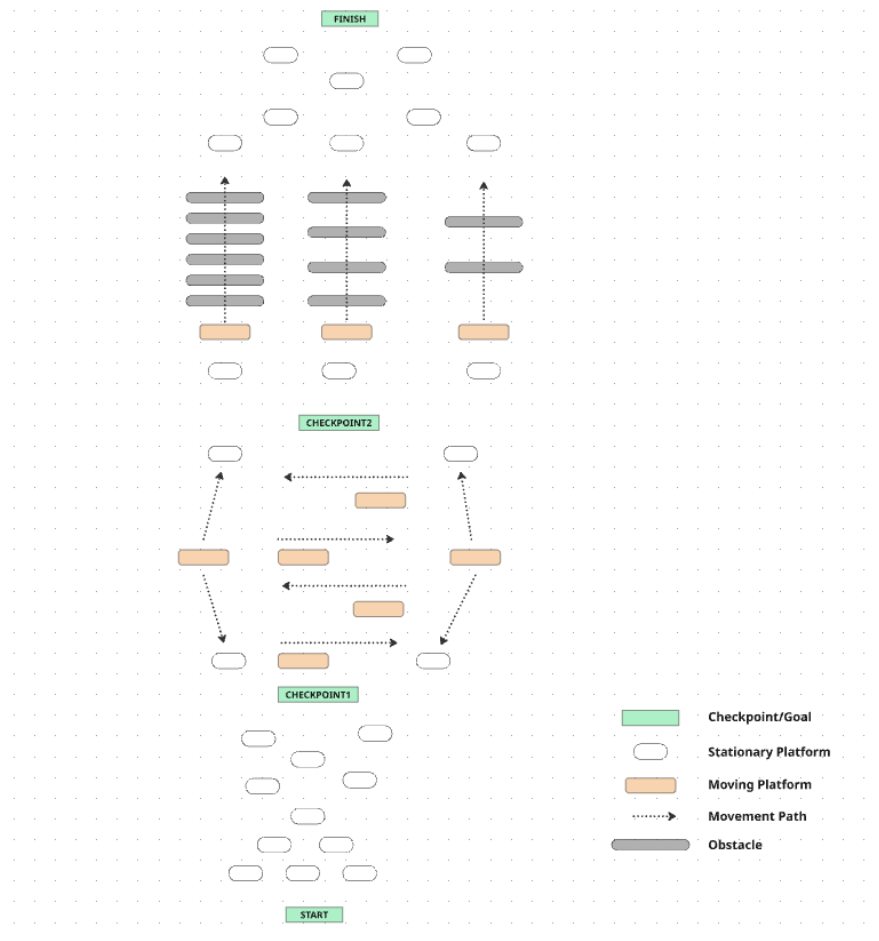


Figure 3.3: Schematic top-down representation of the static level

The first segment of the level was the easiest since it is used to introduce core movement mechanics to the player. The beginning part contained straightforward static platforms with small gaps, these can be crossed with simple single jumps without immediately requiring precise timing. This section was intentionally kept simple so the players can have a basic understanding of the movement and also understand the jump distance and landing behaviour.

As the first segment progressed, the platforms increased in the vertical gap. These later jumps required the use of double jumps to reach higher positions

along the z-axis. The main reason to add this was to teach both the fundamental jump types in the first part of the level before adding moving platforms later on that will require both jumps to progress. Instead of explaining everything through additional tutorial text, the level geometry was placed in a way that the players understood what type of jumps to use to progress. The mechanics were introduced in a manageable context before combining them into more difficult challenges.

The second segment takes a leap in difficulty with the focus shifting from basic movement to timing. In this section, a large portion of the level is platforms that move continuously along a predefined path. The movement was either horizontal or vertical or in some instances could be both which means the participants had to judge not only the distance of the jump but also what position to aim for that makes them land on the platform. Sprint jumping becomes very useful in this section since it provides more leeway even if the player jumps at the wrong time.

The moving platforms were implemented using Unreal Engine's built-in interpolated movement component which can assign control points to each platform and also give each platform its individual moving speed. This was done because it provided predictable and repeatable platform behaviour with minimum code complexity. The implementation itself was intentionally simple since the purpose of the static level was not to demonstrate a complex platform-control system. Most of the design work instead focused on platform placement, spacing and timing.

The third segment was the most difficult portion of the level since it really requires the players to master the movement and have a good understanding of how to cross moving platforms. This segment also has an interesting addition to the level. The player was provided three different paths to reach the next checkpoint where each path has its own positives and negatives.

The left path contains more obstacles but the moving platform itself is very slow so the player will have more time but need to be quick with the jumps. The middle path uses a more moderate number of obstacles and a more standard movement speed. The right one on the other hand has less obstacles but the platform moves very quickly so the timing matters the most. Players were free to choose any of the three routes. This design was included so that the static level still had some sort of route choice rather than forcing every participant through one completely

identical path. However, the routes still remained fully predetermined by the designer. This distinction was important for the comparison with the dynamic level where the player creates their own path.

The third segment acted as a final application of the movement system rather than a tutorial segment. At this point, the participants were fully aware of all the movement techniques, timings and all they were made to do is master these skills and pick the route that is most suitable for them. Its increased complexity was deliberate and followed the same progression principle used throughout the level.



Figure 3.4: Top-down view of the three routes in the third segment of the static level.

All the platforms in the static level used the same cloud-based visual style as the dynamic level. This helped maintain the visual consistency between both the conditions. The main experimental distinction was based on the platform behaviour and environment control.

3.1.3 Dynamic Cloud System

The dynamic level was built around a cloud system where the objects can change their traversal logic in response to player input. Instead of asking the players to perform jumps across platforms placed by the designer, the dynamic condition

asked them to decide how each cloud should behave before using them. Two cloud types were used: the base cloud and the breaking cloud. The players can interact with the base cloud and change its functionality while the breaking cloud is a dynamic cloud that behaves uniquely when the player lands on it. The base clouds are supposed to be direct player control over the environment while the breaking cloud is more of a reactive environmental behaviour that the player had to respond to.

The base cloud has three possible states: Static, Moveable and Bouncy. In its default static state, the cloud works like a normal platform that is rigid. Starting the level on a standard state was important because it gave the players a familiar baseline and made the effect of later state changes more noticeable. Players selected the cloud states through the hotbar that was mentioned earlier in the player controls section.

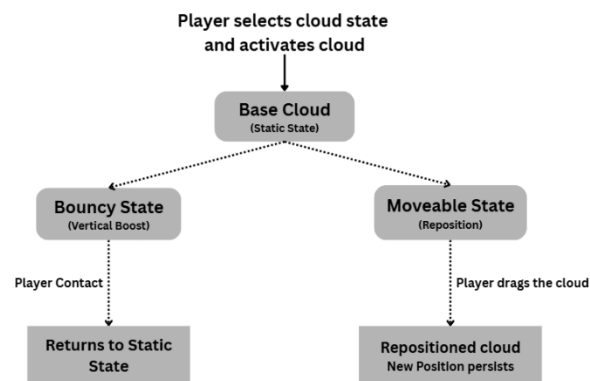


Figure 3.5: Visual representation of the dynamic cloud system.

The bouncy state was designed to help the player reach a higher vertical distance by modifying the environment rather than the character movement itself. Once activated, the cloud stays bouncy until the player lands on it. When the player comes in contact with it, it will launch the player upward and then return back to static state. This is done so that one interaction does not permanently change the cloud to just one state. Instead, the players have to decide when and where the bouncy state would be the most useful and apply it to that particular cloud platform. The bouncy state also had visible animations to indicate a particular cloud is in that state by constantly wiggling like jelly. This feedback was important since this study only focused on how many cloud interactions each player has, and not finding if players guessed which cloud is in a bouncy state. Providing a

clear visual response reduced ambiguity and helped ensure successful or failed traversal attempts were tied more closely to player decisions and movement execution.

The moveable state was designed to give the players the ability to reposition a cloud and create their own route through the level. When selected, the targeted cloud can be dragged across the horizontal XY plane but not vertically. Since the bouncy state already exists for the Z axis, this restriction was placed so that all the states have their own unique purpose. The movement of the cloud was also limited to a maximum radius from its initial position. A circular material was created around the cloud to show how far it can be moved to. This helped the players understand their spatial limits clearly when interacting.

```
void APlayerCharacter::TryMoveCloud(float DeltaTime)
{
    if (!IsDraggingCloud || !IsValid(moveCloud) || cloudType != ECloudType::Moveable)
        return;
    APlayerController* PC = Cast<APlayerController>(GetController());
    if (!PC) return;
    float MouseX, MouseY;
    PC->GetMousePosition(MouseX, MouseY);
    FVector2D CurrentMouse(MouseX, MouseY);
    FVector2D MouseDelta = CurrentMouse - LastMousePosition;
    LastMousePosition = CurrentMouse;
    if (MouseDelta.IsNearlyZero()) return;
    float Sensitivity = 3.0f;
    FVector CamForward = camera->GetForwardVector();
    FVector CamRight = camera->GetRightVector();
    CamForward.Z = 0.f; CamForward.Normalize();
    CamRight.Z = 0.f; CamRight.Normalize();
    FVector WorldMove = (CamRight * MouseDelta.X * Sensitivity)
        + (CamForward * -MouseDelta.Y * Sensitivity);

    FVector NewLocation = moveCloud->GetActorLocation() + WorldMove;
    NewLocation.Z = moveCloud->GetActorLocation().Z;
    FVector Offset = NewLocation - cloudStartLocation;
    Offset.Z = 0.f;
    if (Offset.Size() > cloudMoveRange)
    {
        Offset = Offset.GetSafeNormal() * cloudMoveRange;
    }
    NewLocation = cloudStartLocation + Offset;
    NewLocation.Z = moveCloud->GetActorLocation().Z;
    moveCloud->SetActorLocation(NewLocation);
}
```

Figure 3.6: C++ Function used to reposition the moveable cloud.

Once the current mouse position is collected, it is then converted into world-space movement for the camera. The cloud's Z position is kept fixed to its original

position. Before applying the new cloud position, the offset is checked if it surpasses the cloud range and if it does, it is clamped to the highest position possible and then cloud position gets updated.

Similar to the bouncy state, only one cloud can be moved at a time and also the player is not allowed to move the cloud that they are standing on. These constraints were applied to stop the mechanics from becoming simple direct player transportation to the next checkpoint. By adding these minor rules to how the clouds behave in each state, the core principle is still present which is to allow the players to interact with the environment and use the environment to progress in a platformer level. Unlike the bouncy state, a moved cloud remained in its new position for the rest of the run unless the player repositioned it again. This is done so that even if the players die after constructing a route, they can still go back and reuse the same one or change it.

The prototype also implemented a separate breaking cloud. This cloud was visually distinct from the other clouds and had its own state sequence when the player interacted. There are 4 states: Whole, Warning, Breaking, Reforming. When the player makes contact with the cloud, it first enters a warning state which provides a visual indication that it will break. Once the warning state ends, the cloud breaks into 4 small fragments and these fragments start drifting away from the initial cloud position. After a short delay, these fragments will move back towards the initial position and combine back into the original cloud object. Unlike the base cloud, the breaking cloud did not ask the player to select a state, but instead added pressure and instability to the traversal space.

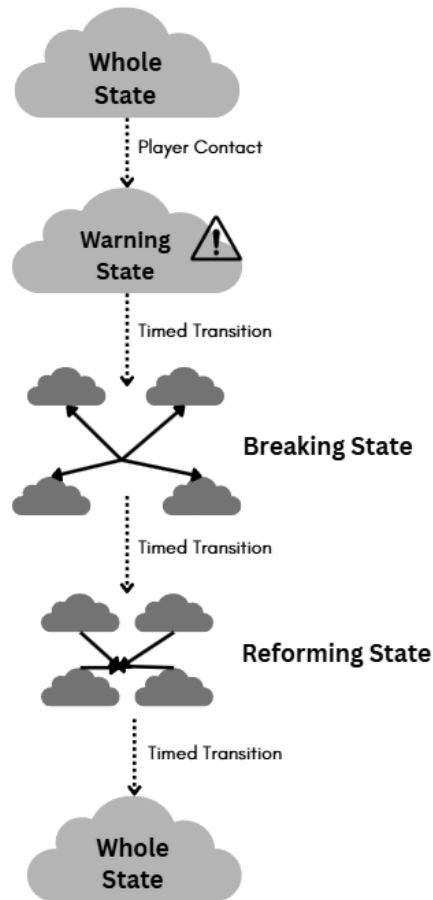


Figure 3.7: State breakdown of the breaking cloud.

The multi-state design was chosen because it was much more readable and interesting compared to making it disappear immediately. The warning state gave the player a brief moment to react and let the player stay on of the fragments if wanted or jump towards another platform that is close by. The breaking cloud is not affected by the state changes that are normally applied to base clouds.

The cloud system was implemented with both blueprint logic and also C++ structures and interfaces. Blueprints were used for the state transitions and interactive behaviour while C++ was used to communicate what the player did with the targeted cloud. This provided a balanced flexibility in gameplay iteration with enough structure for the interaction system to remain manageable during development.

3.1.4 Dynamic Level

The dynamic level was designed as the experimental condition of the study and uses the same three-segment progression that static level used. This made sure that both the conditions have the same structure and it's easier to compare them and find the differences. The player character, movement logic, checkpoint system are all the same as the static level with the addition of the dynamic cloud system described in the previous subsection. The dynamic level also had a deliberate increase in environmental demand across the three sections but this was not achieved with the help of moving platforms instead there are less platforms to work with as you progress in the dynamic level and required greater use of cloud manipulation.

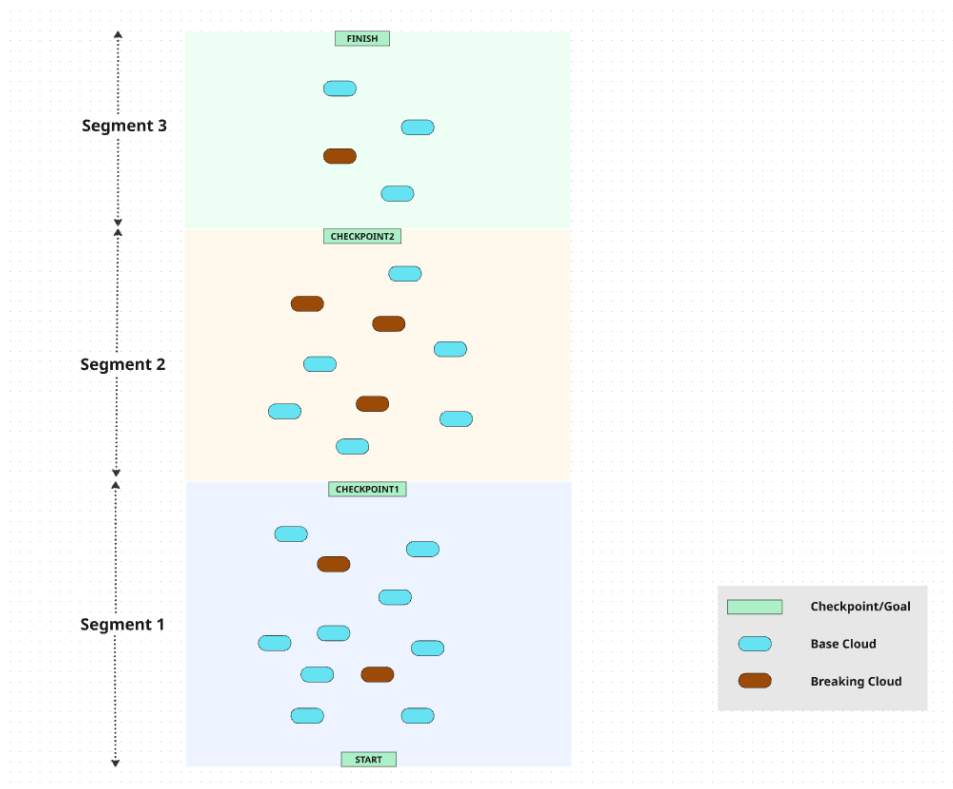


Figure 3.8: Schematic top-down representation of the dynamic level.

The first segment was designed as a relatively forgiving introduction to the dynamic cloud system. There were a large number of cloud platforms throughout the section, creating various possible landing opportunities. This gave the players a chance to learn how to aim and change cloud states. At the same time, the first segment was not designed around a single fixed route and in fact there are

multiple different ways to complete it because of the vast number of clouds available to the player. The first segment also introduced the breaking cloud. Its distinct appearance helped separate it visually from the base clouds. Including this mechanic early lets the players experience how it works without having to lose any progress.

The second segment reduced the number of available platforms and increased the importance of environmental interactions. Unlike the first segment, the original placement of the clouds alone will not help the players complete this segment. Participants have to make use of the bouncy, moveable and breaking clouds to create a viable path towards the next checkpoint. While segment 1 was used to introduce the cloud mechanics, segment 2 is an actual test to see if the players understood these dynamic clouds and applied them instead of simply jumping across. No route was deliberately presented as the correct or easiest solution. This was important because introducing an obvious route would reduce the opportunity to observe the player's traversal strategies.

When the players reach the third segment, they already encountered the control system, experimented with the cloud states and made their own path till this point. The final segment therefore removed more of the immediately usable traversal structure and required the players to rely more heavily on their own interpretation of the environment. So, the third segment acted as a form of mastery section for this level similar to static level.

Overall, the dynamic level was designed to preserve the core platforming demands of movement, jumping and landing while adding the environmental decision-making as an additional layer. This level provided the experimental contrast needed to examine whether control over the environment changed the player behaviour but also their routes, pacing and strategies.

3.2 Experimentation

3.2.1 Gameplay telemetry

Gameplay telemetry was collected automatically during both the static and dynamic conditions using a custom data collection subsystem implemented in Unreal Engine 5. Automatic logging is the most consistent way to collect data from the playtest without requiring manual observation or note-taking. A central `DataCollectionSubsystem` manages the logging process across each gameplay

session. At the start of each session, the subsystem stores the player ID which is important to differentiate all the play testers. This playerID is a series of numbers that is unique to each player. It also identifies the level that is being played and saves it in the level type slot. Every playthrough, the subsystem erases the previous data that was stored and starts a new save for the logs. At the end of the session, total completion time, total jumps, total deaths and total cloud interactions were all calculated and stored at the top of the log file.

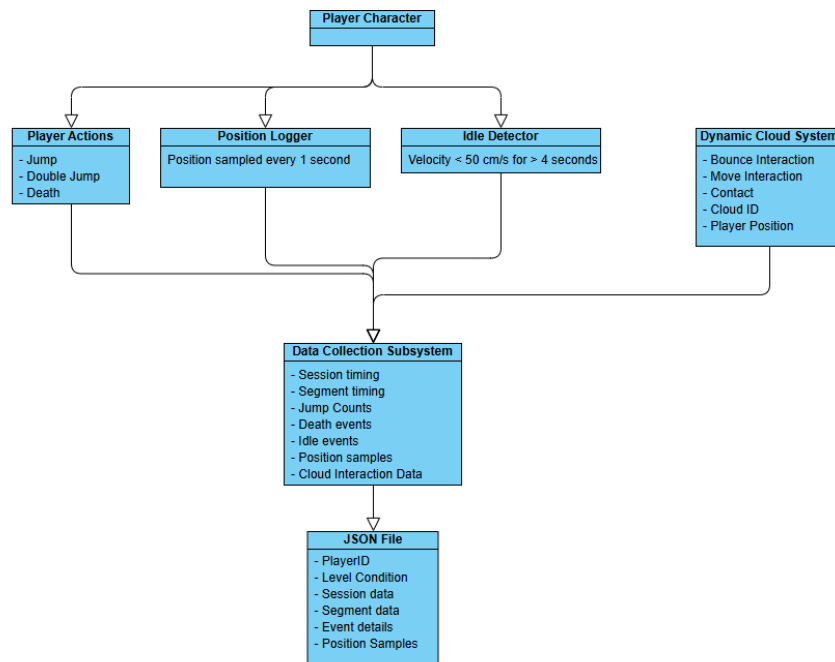


Figure 3.9: Class diagram for Gameplay Telemetry.

Since the levels are divided into three segments each respectively. The logging system takes advantage of this and calculates the data for each segment as well. So, it calculates the number of deaths, jumps, idle events in each segment which helps the data analysis process. Jump behaviour was recorded directly from the player character. Normal jumps and double jumps were stored separately in each segment. This distinction was included since both actions represent different traversal choices. While jumps were counted together and saved as a single variable/unit, deaths were recorded as individual events. So, each death stored the elapsed timestamp, the segment in which the death happened and also the position at the point of death. The position data for each data later allowed to plot

the death locations on a graph which gives a visual representation of the deaths in both static and dynamic levels clearly.

Idle behaviour is also recorded because small periods of no movement provide information regarding the changes in traversal pacing. A `IdleDetectorComponent` was created which is attached to the player character, it checks the player movement while the character is on ground. An idle period is detected when the player movement velocity is below 50 cm/s for more than four seconds. When this threshold is reached, the detector automatically adds the idle data to the subsystem. Each segment has its own Idle Events as well. Similar to the Idle Detector, a `PositionLoggerComponent` was also added to the player character which is simpler than the idle detector since it just logs the player position every second. The player position data is vital to the research since it can be visualised to show the traversal paths.

The dynamic level required additional telemetry since the participant interaction with the clouds was one of the main behaviours being investigated. Each cloud interaction stored the cloudID, current cloud state, interaction type, timestamp and segment number. The interaction types distinguished between bounce, move and contact events. This data is collected from the base clouds and the breaking clouds. Whenever a cloud state changes, it sends the data to the subsystem which will store it in a cloud data array.

At the end of each playthrough, the collected data were automatically serialised into a JSON file. This file contained session metadata, segment data, cloud interactions, death events, idle events and sampled player positions. Files were safely saved to the project's directory. Every participant is later guided on how to find these files and submit the files for data analysis.

3.2.2 Playtesters

A total of 10 participants took part in the playtesting study. Participants were recruited through personal invitations and other university students from Abertay University Esports Discord Server who were available to take part in the experiment. The study did not try to recruit players according to their skill levels, since the aim was to see how the players with different levels of gaming experience reacted in the two platforming conditions.

The questionnaire was also used to collect basic indication of the participant experience with video games. This provides an important context of how often the players play video games and can support their final completion times. Other than their gaming experience, no other demographic variables were collected. Each participant completed the static level first and the dynamic level second. The same order was used for every player so the playtest remained consistent across the sample. All the players have completed the playtest remotely with the help of the designer for troubleshooting any technical issues. All the players were also provided an instructions file which gave them the general context of the playtest. The end goal they need to reach which is the finish point. Everything else was informed in the prototype itself.

All the participants completed both the level conditions before answering the questionnaire. Gameplay data was automatically recorded and the players were guided where to find the log files in the questionnaire. Participants then completed the questionnaire based on their experience of both levels, allowing them to directly compare the two traversal approaches.

3.2.3 Questionnaire

After completing both static and dynamic levels, participants completed a questionnaire designed to capture their perception of the static and dynamic levels. The questionnaire was used alongside gameplay telemetry because behavioural data alone could show that participants took longer or followed different routes but cannot explain why a player took a certain decision. So, the questionnaire provides more subjective data by each player on level understanding, interactions and enjoyment.

The questionnaire is organised around several themes and the first theme was perceived difficulty. The participants were asked if they thought the dynamic level was harder than the static level. This provided a subjective difficulty compared with their completion time and deaths. Another theme was their understanding of the cloud mechanics since it is so essential to know what the dynamic clouds do to complete the dynamic level. Participants were also asked whether it felt like

they were interacting with the environment or solving a puzzle in the dynamic level.

Most of the closed questions used a five-point agreement scale from strongly disagree to strongly agree. A consistent Likert-style format was used to make responses easier to compare across players. The statements focused on the main areas that are relevant to the research question. Short open-ended questions were also included after selected statements. These responses were included because agreement ratings alone could not explain why each player picked a certain option. This questionnaire provided the required data to study the player perception.

3.3 Data Analysis

The collected data were analysed using a combination of descriptive statistics, spatial visualisation and questionnaire analysis. These methods are used to find out whether environmental state changes affected observable traversal behaviour and whether players perceived the dynamic level differently compared to static level. Since every player completed both the static and the dynamic levels, their datasets can be considered as paired observations. Descriptive statistics are calculated for all the variables from the data where mean, standard deviation are calculated to describe the variation between participants.

Spatial traversal behaviour was examined with the help of a python script developed using NumPy and Matplotlib. Raw position data has too many closely spaced coordinates, so successive positions were removed unless the player moved at least 80 units from their previous selected position. This reduced the visual clutter caused by the repeated samples from time when the player moved very little. Position data where the z axis is below -500 units is also removed since that is the indication that the player died from falling below the playable level.

Smoothing was also used to make broad route differences readable rather than displaying every small positional data.

```
Input: Position samples
Output: Smoothed X-Y traversal path

1. Set startX to first valid player X position.
2. For each position sample:
    a. Normalise X using:  $X = X - \text{startX}$ .
    b. Ignore sample if  $Z \leq -500$ .
    c. Ignore sample if movement from previous retained point is less than 80 units.
3. Store remaining X and Y values.
4. Smooth X and Y using a moving average with window size 18.
5. Plot the smoothed X-Y path.
6. Add level segment boundaries.
```

Figure 3.10: Algorithm to visualise traversal paths.

Death-location plots used X-coordinate normalisation which allowed horizontal progression to be compared visually without differences in coordinates. Instead of using the death position directly, the script instead located the previous position where the z axis is on the playable units and used that position to indicate where the failed traversal attempt originated.

```
Input: Death events, position samples
Output: X-Y death-location map

1. Set startX to first valid player X position.
2. For each death event:
    a. Normalise death X using:
        $X = X - \text{startX}$ .
    b. If death Z > -500:
       use recorded death X and Y.
    Else:
       find the last valid position
       before death where Z > -500.
       use that X and Y instead.
3. Plot each processed death position.
4. Add level segment boundaries.
```

Figure 3.11: Algorithm to visualise death locations.

Finally, the closed questionnaire was summarised using response counts across the five agreement categories rather than treating the individual Likert responses as continuous measurements.

Chapter 4 Results and Discussion

In total, 10 participants have played the game prototype and filled the questionnaire afterwards. These participants were recruited through personal invites and community invites as well. All the players were guided thoroughly about the study and details about the prototype itself.

4.1 Overall Traversal Performance

The overall gameplay comparison showed clear differences between the static and dynamic levels, particularly in completion time and idle behaviour. Table 4.1 summarises the main telemetry differences between the two levels.

Metric	Static (Mean)	Dynamic (Mean)	Difference
Total Completion Time (s)	331.2	425.1	+93.9
Total Jumps	98.4	87.6	-10.8
Total Deaths	22.4	20.2	-2.2
Total Idle Events	3.9	24.7	+20.8
Total Idle Duration (s)	12.9	87.2	+74.3
Cloud Interactions	N/A	53.1	--

Table 4.1: Summary of gameplay telemetry for static and dynamic levels.

The average completion time increased from 331.2 seconds in the static level to 425.1 in the dynamic level which is a 93.9 seconds' increase. This is a clear indication that the players took longer to complete the dynamic level. But the number of jumps taken reduces by 10.8. However, jump count alone cannot be used to indicate that the players had more successful jumps in dynamic level. The dynamic mechanics could change the number of jumps required. So, the jump

result is treated as a descriptive difference rather than evidence of improved movement performance.

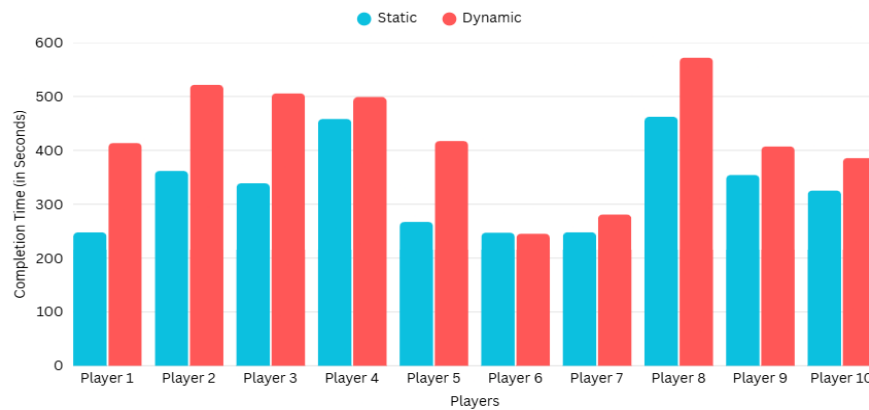


Figure 4.1: Static and Dynamic completion times by participants.

Figure 4.1 shows that most players that participated in the playtest spent more time completing the dynamic level and the data is not inflated because of one unusually long playthrough. Player 6 is the only exception who actually ended up taking less time to complete the dynamic level compared to the static level. This increase in completion time was not accompanied by an increase in deaths. Players recorded an average of 22.4 deaths in static and 20.2 deaths in dynamic. This suggests that increased death frequency was unlikely to be the only reason players took longer to complete the dynamic level.

The clearest behavioural difference can be seen in the idle activity. Average idle duration increased by 74.3 seconds in dynamic level, while the number of idle events also increased substantially. Figure 4.2 will show the differences across individual participants.

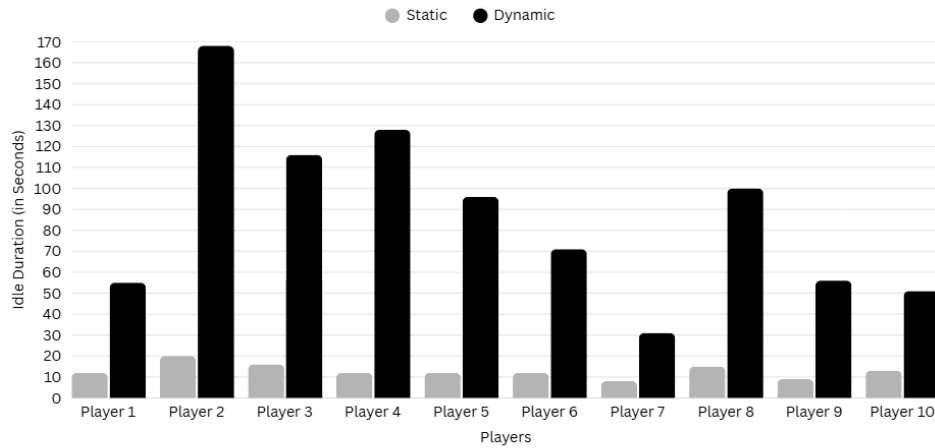


Figure 4.2: Static and Dynamic idle duration by participant.

This player pattern suggests that the pacing of traversal changed in dynamic condition. In the static level, the player mainly focused on where the player is and how to time their movement. In the dynamic level, the player needs to inspect the clouds nearby and come up with a path they think will work while also aiming at clouds, selecting the desired states. These interactions created more opportunities for the player to stop before continuing.

Idle events alone cannot confirm that every pause or break the player took is for planning. Sometimes the player can stop because of uncertainty. But, with the help of the questionnaire a clearer conclusion can be drawn later. Just from the gameplay telemetry, data suggests that the players do pause more often in dynamic level. However, there were no supporting evidence for these idle events that suggest that it was clearly for decision making.

4.2 Segment-Level Traversal Differences

Looking at the levels as three separate segments provided a more detailed view at where the difference in overall completion time comes from. In Segment 1, the average completion time increased from 48.1 seconds in static level to 78.0 in dynamic level. Both the levels used the first segment as an introduction to the levels where static level used it to teach the movement abilities while the dynamic level teaches the players how to change the cloud states. Although the dynamic level gives more freedom to the player, the participants might have taken additional time to get used to the interaction system and movement controls.

Segment Data	Static	Dynamic
Segment 1 Completion time (s)	48.1	78.0
Segment 2 Completion time (s)	79.8	201.8
Segment 3 Completion time (s)	203.3	145.3

Table 4.2: Segment data for static and dynamic levels.

The largest difference between the two levels appeared in Segment 2. Players spent approximately 122.0 seconds more in dynamic level. While the static level did increase moving platforms, they can be tackled with brute force. Dynamic level reduced the number of usable clouds and environmental manipulation became more important. One possible explanation is that the players required additional time to evaluate cloud states and create a viable path. However, this could have also resulted from greater unfamiliarity with the cloud layout or difficulty operating the cloud platforms. A different pattern appeared in segment 3 where the static level took longer to complete compared to the dynamic level. Both the segments were designed as mastery sections. The static level introduced three pre-determined route options which could made it difficult to pick a path. This does mean that the dynamic level did not produce consistent increase in completion time across the entire level.

Overall, the segment-level comparison shows that the longer dynamic completion time was concentrated mainly in the first two segments. This suggests that the effect of environmental state changes depended partly on how strongly each section required the players to use and plan around the mechanics.

4.3 Deaths and Traversal Failure

Across all ten players, the static level recorded 224 deaths, compared to 202 deaths in the dynamic level. This is useful when interpreting the longer completion and idle times discussed earlier, since it indicates that participants were not simply spending more time in the dynamic condition because they were repeatedly failing jumps. The death-location visualisations provide additional information by

showing where these failures occurred rather than only reporting their total frequency.

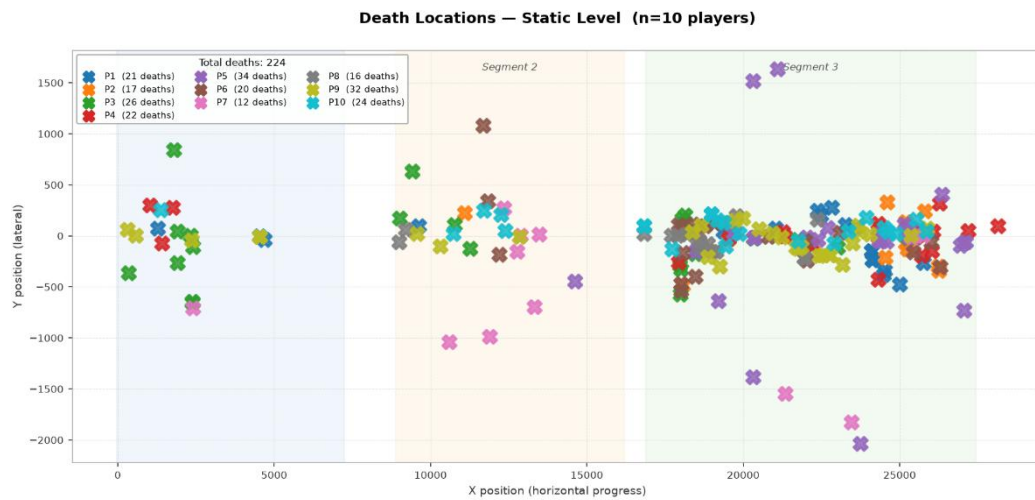


Figure 4.3: Death locations recorded in the static level

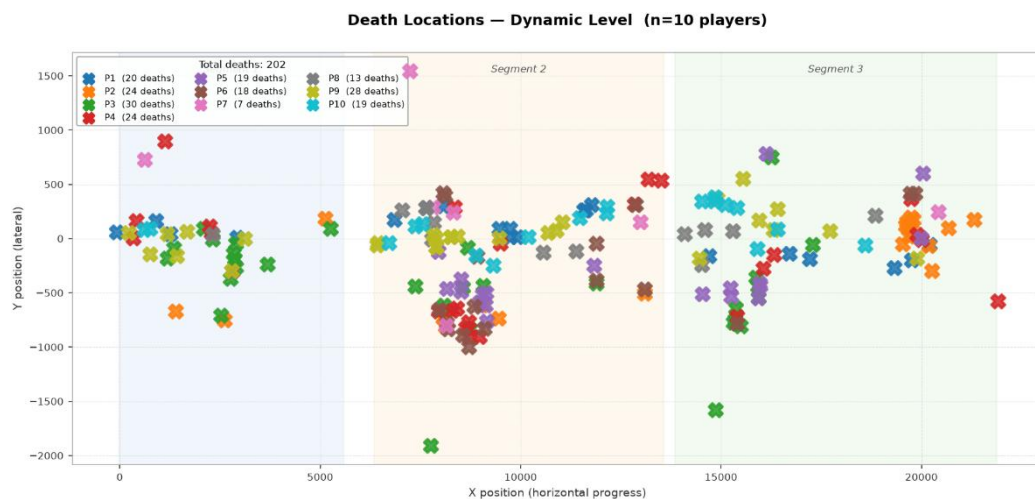


Figure 4.4: Death locations recorded in the dynamic level.

Death locations were distributed throughout the level in both static and dynamic conditions instead of being limited to a single location. In static level, players died more in the third segment compared to segment 1 and 2 combined. This could indicate that mastering the static level can be tricky. There is another interesting finding from the static level graph, most players did not even take the other two routes and stuck to the middle route itself. Only 2 players have attempted the other routes which suggests that the middle route seemed more direct, visible or attractive to most participants. This could mean that the other two paths didn't give the players enough reasons/advantages to pick them over the middle path.

The dynamic level produced a different type of traversal challenge. From the graph, Segment 2 seems to have the most deaths which could be related to the reduced number of usable clouds and higher requirement for environmental manipulation. However, this could also be the particular cloud placements by the designer which could have made it harder for the players to interact with. Death location data only provides where failure occurred but does not establish the underlying cause. The death results suggest that the dynamic condition shifted some of the challenges away from repeated mechanical failures and towards traversal decision-making and environmental manipulation before the movement was even attempted.

4.4 Dynamic Cloud Interaction

The Dynamic level introduced several interaction types that were recorded during gameplay. Across all the players, the telemetry recorded 241 Move interactions, 166 Bounce interactions, and 124 Contact interactions with breaking clouds. These values show that the players were actively interacting with the environment mechanics instead of progressing through them with movement mechanics alone.

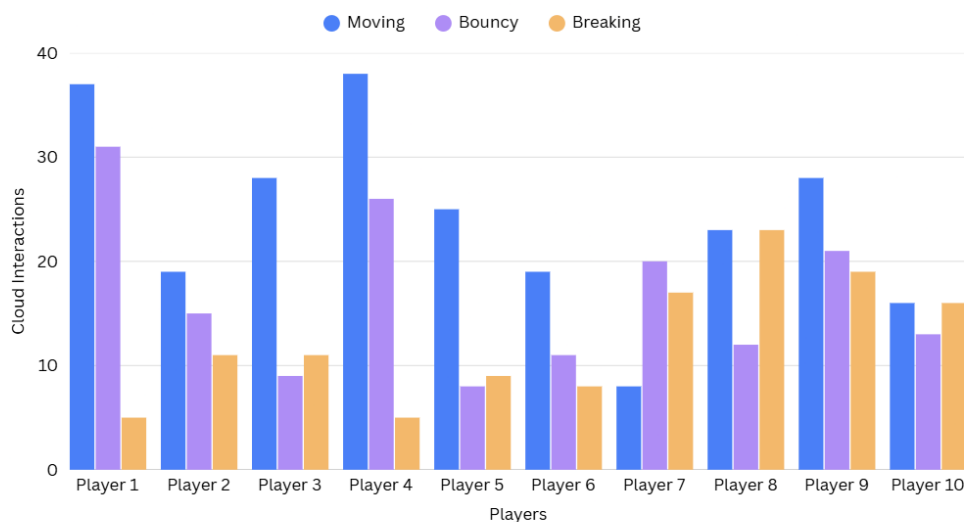


Figure 4.5: Dynamic cloud interaction by participant and interaction type.

From the Bar graph, the move interaction is clearly the most frequently used cloud state by the players. This is expected since the dynamic level design did indicate the requirement of repositioning clouds to create the best route possible. Bouncy interactions were also used regularly to cover greater distances both in Z axis and

XY axis as well. Contact events on the other hand were used less by most players but a few individuals used the contact event with the breaking clouds more than the other two state interactions. The participant-level interaction bar graph is useful since it provides detailed information about each player's behaviour instead of relying on overall totals. These differences indicate the different traversal strategies but they alone cannot be used to establish player preference. Although, the questionnaire does ask the player for their dynamic cloud state preference which can be seen later. Some mechanics may also have been used more often because of the way particular sections were designed or because one interaction required repeated activation.

4.5 Traversal Routes and Spatial Behaviour

The traversal-path visualisations were used to compare how participants moved through the static and the dynamic levels. Unlike completion time or death counts, these graphs provide a spatial view of the routes taken through each condition and show how much variation existed between players. These path visualisations provide a clear view at how each player tackled both the levels. In particular, the dynamic level visualisation is important since this study expects the players to make their own paths.

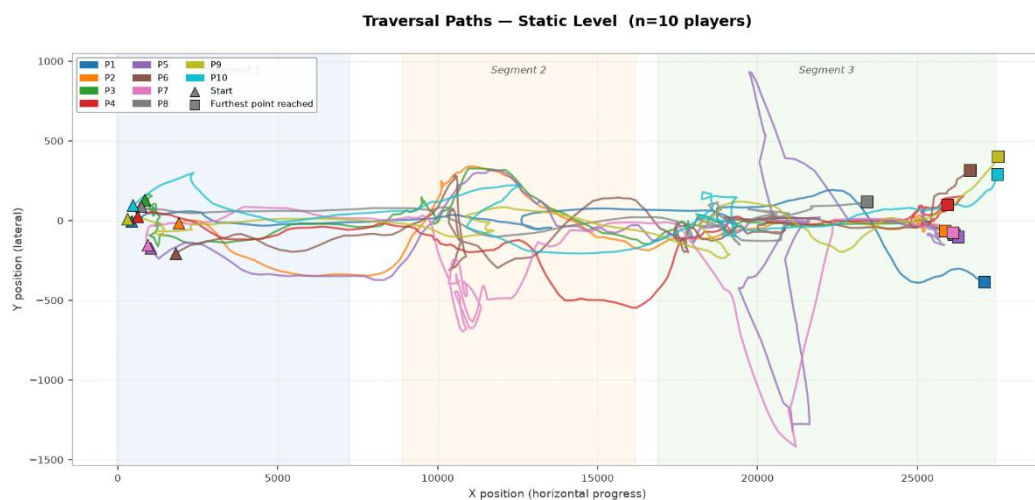


Figure 4.6: Simplified traversal paths from the players in the static level.

The static level shows several areas where every participant path overlaps very closely, particularly through sections where the available platforms strongly

directed progression. However, there is a minor route variation. Both P4 and P7 attempt to take the right side of the level during segment 2 while a few others attempt to complete the segment from the left side of the level. Similarly, Segment 3 follows the same pattern that was found during the death location analysis where most players did not take the other routes and stuck to the middle path while 2 players attempted the other routes as well.

Most players followed the central route despite the presence of alternative paths in the static level. This indicates that the central path has been perceived as the most obvious path. However, the data does not reveal whether the players have failed to notice the alternative paths or considered the other routes not worth their time.

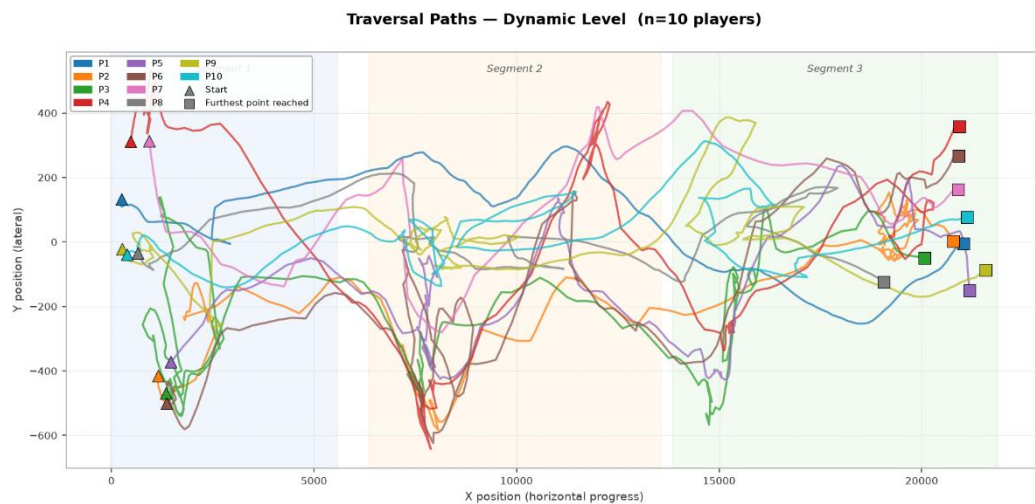


Figure 4.7: Simplified traversal paths from the players in the dynamic level.

The dynamic level shows a clear variation in traversal paths between players even though the level was the same for all the players at the start. Rather than choosing between routes established during level design, participants could alter local traversal possibilities through bouncy and moveable cloud states. Most players were able to create their own path to complete each segment. Player 4 showed particularly high route variation and changed direction several times before reach the goal.

Even though both the levels have different paths. The players have clearly ignored these paths in the static level while actively making their own path in the dynamic level. This difference supports the central comparison of the study. Player-driven

environment state changes did not replace static platforming choices but introduced an additional form of traversal control by allowing the players to influence the level they were navigating.

4.6 Player Perception

The questionnaire was used to examine how participants perceived the dynamic condition after completing both levels. Table 4.3 summarises the responses for each participant and includes a final row showing the most common response for each question.

PlayerID	Gaming Frequency?	Dynamic Harder than Static (1-5)	Understood Cloud Functions (1-5)	Clouds Felt Interactive (1-5)	Route Freedom (1-5)	Navigation Changed (1-5)	Dynamic More Enjoyable (1-5)	Most-Used Cloud
Player1	Every day	3	4	4	5	4	4	Bouncy
Player2	Every day	4	4	4	3	2	3	Moving
Player3	Every day	5	5	4	5	5	5	Moving
Player4	Every day	4	5	4	4	4	4	Bouncy
Player5	Every day	4	5	4	5	4	4	Moving
Player6	Once a week	2	5	5	5	4	5	Breakable
Player7	Every day	4	5	4	4	5	4	Bouncy
Player8	Every day	4	4	5	5	4	5	Moving
Player9	Once a month	2	3	4	4	4	4	Bouncy
Player10	Once a week	4	5	5	5	5	5	Moving
Mode:	Every day	4 (Agree)	5 (Strongly agree)	4 (Agree)	5 (Strongly agree)	4 (Agree)	4 (Agree)	Moving

Figure 4.8: Table representing the participant questionnaire responses.

Perceived difficulty produced a more mixed response than the other questionnaire items. 7 of the ten participants agreed or strongly agreed that the dynamic level was more difficult than the static level. This supports the gameplay telemetry showing longer completion times and more idle behaviour in the dynamic level. But as discussed earlier, this resulted in less deaths as well. The questionnaire therefore establishes that most players felt the dynamic level was harder but does not identify a single reason for this perception.

Understanding of the dynamic cloud system was generally high. This is important since the behavioural differences observed in the dynamic level are less likely to

have resulted from participants not understanding the mechanics. The open responses also indicated that participants commonly used the instruction screens and some experimentation during the first segment to understand the cloud behaviour. The strongest responses were related to environmental interaction and route freedom. Since all 10 participants agreed that the clouds felt like something they were actively interacting with rather than figuring out obstacles. 9 participants also agreed that they felt free to approach the dynamic level in their own way. These responses indicate that the players generally felt the cloud mechanics as active environmental interactions and felt that they had more freedom in the traversal.

Players were also asked if their navigation approach changed from static level to dynamic level. 9 of the participants agreed that their approach has shifted while one of them disagreed. One of the written responses said "In the static level, I mainly stuck to one path I found the easiest to navigate since most of the jumps were pretty straightforward.". They also said the dynamic level has so many paths and with each path having its challenges. They mentioned it is wise to analyse all the paths before picking one since some of the paths are hard to switch once some progress is made. This could also be evidence towards why dynamic level has high idle duration and idle events.

Participants also responded positively to the overall experience of the dynamic level. 9 players agreed or strongly agreed that the dynamic level was more enjoyable than the static level. This suggests that the players can find levels hard but also have fun at the same time. The two findings are not necessarily contradictory. Players were also asked what cloud type they believed they used the most, 5 of the players selected moving, four said bouncy while one individual selected breakable. This matches well with the interaction telemetry since move interactions were the most frequently recorded cloud state change.

Overall, the questionnaire suggests that the players understood the dynamic cloud system. Also, most players considered dynamic conditions to be more difficult while also saying they had more fun as well which indicates that the additional environmental control changed both how the level was approached and how the experience was perceived.

4.7 Overall Findings

The results show that there are differences between the static and dynamic levels, but with such a small sample and different level layouts, these differences need to be interpreted with caution. The Dynamic condition took longer for participants to complete, and resulted in more idle activity, but the number of deaths did not vary greatly between conditions. This pattern is consistent across all the participants with the dynamic level giving a slower traversal style but the telemetry does not provide any insight into whether this was due to planning, uncertainty, complex dynamic interactions or some mix of these.

They also indicate that the effect was not uniform across the level, as revealed in the segment results subsection. Dynamic completion time was the highest in Segment 2, and participants on average were faster in the segment 3 of the dynamic condition. This implies that manipulation of the environment did not cause slower traversal in all segments. The patterns observed at the segment level might be due to a number of different factors, including variations in the movement mechanics familiarity, level structure and dynamic cloud interactions.

The spatial visualisations and the cloud-interaction telemetry offer descriptive evidence of the different ways in which the participants used the Dynamic environment. There was more variation in Dynamic traversal paths and participants interacted with Move, Bounce and Breaking Clouds in varying combinations. The two levels, however, did not have an identical level design, so the path variation observed cannot be solely attributed to the dynamic environmental states' mechanic.

The questionnaire is used to capture further insights into the experience from the perspectives of the participants. In the Dynamic level, most reported increased route freedom, along with a change in navigation approach and increased enjoyment but also found it to be more challenging. The responses give evidence to support the hypothesis that environmental manipulation influenced the understanding and traversal approach. Concurrently, the questionnaire is subjective in nature, and possibly affected by novelty, previous experience of the platformer games and the order in which the levels were played.

Overall, the study offers some preliminary evidence that player controllable environmental states can affect traversal behaviour and player perception in a 3D platformer. The findings are not strong enough to establish a general causal relationship, but they identify patterns in pacing, route variation and perceived freedom that justify further investigation with a larger and more controlled study.

Chapter 5 Conclusion and Future Work

This study investigated how player-driven environmental state changes affect player behaviour and player perception in a 3D platforming game when compared with static platform behaviours. A game prototype was created with two level conditions: a static level with predetermined platforms and level design and a dynamic level where the player can alter the level and the platform positions and their functions through moveable, bouncy states, along with breaking clouds which have their own dynamic behaviour. Ten participants have played both the levels from which gameplay telemetry was collected and they have answered a questionnaire which together gives this research an opportunity to study and compare behaviour and experience.

The obtained data were analysed to identify some patterns. There was a general trend for participants to take longer to finish the Dynamic condition and to have more idle time, but the overall number of deaths was comparable across the two conditions. One possible explanation is that environmental manipulation imposed extra periods of route evaluation/planning prior to movement. The study did not measure cognitive decision making directly; however, the pauses could have been due to uncertainty, interaction time or difficulty understanding what to do next. Comparisons between level segments also yielded mixed results. Dynamic completion time was higher in Segments 1 and 2, with the largest difference occurring in Segment 2, but the Static condition was slower in Segment 3. This turnaround might have been driven by familiarity with the Dynamic mechanics, but alternative factors include layout and route structure. Therefore, the experiment cannot conclude that manipulation of the environment always accelerates or decelerates traversal time; rather it seems to depend on the circumstances of its use of the mechanic.

The interaction data revealed more variation in the use of the available cloud mechanics in the Dynamic level, and the spatial telemetry revealed more variation in the paths taken by the participants. The results support the notion that player-induced changes in the environment can facilitate different modes of traversal. Some of this variation may have been due to differences in level layout, however, due to the fact that the Static and Dynamic levels were not geometrically equal,

some of this variation may have been due to the environmental mechanics themselves. A more convincing controlled comparison would then be needed before drawing a direct conclusion to the effect that the route variation is entirely player controlled.

Overall, the results of the questionnaire were positive for the Dynamic level. The majority of participants indicated an increase in route freedom, a shift towards a different approach to navigate the level and higher enjoyment levels, with the Dynamic level also being generally seen as more challenging. The general impression of the system was felt to be interactive and flexible by this group of participants as indicated by their responses.

These findings have some limitations. The sample was small (ten) and was largely drawn from personal and university contacts, and so the results may not be widely generalised. Many of the participants also played games with some regularity, so the results might not apply to less experienced players. The experiment was set up so that each subject played Static first, followed by Dynamic, thus allowing for learning and/or order effects. Another restriction is that both conditions had the same general course but not the same level layout. Because of this, the completion time, deaths and spatial paths are not solely due to environmental state manipulation. There were also a number of telemetry measures that were subject to interpretation. Idle duration may be planning, confusion, observation or interaction time and the frequency of interaction does not necessarily reflect mechanic preference. The traversal paths were stylised and regularised to make them easier to read, and therefore should not be interpreted as a true representation of the difference in traversal between the two. Lastly, the questionnaire employed a set of targeted questions that were created for this study.

Future work could address these limitations by testing with a much larger group and also counterbalancing the order of the two levels. Further work could also make the two-level layouts similar so that the dynamic interactions would be the only difference. The dynamic cloud system itself could be expanded more with a variety of mechanics that could persist across the level. Future analysis could additionally examine the traversal behaviour in greater detail. A larger study could

also investigate the relationship between player experience, preferred cloud mechanics and types of routes created.

Despite these limitations, the prototype demonstrates that environmental interactions can provide a meaningful addition to conventional 3D platforming. This provides a good foundation for further investigation into player-controlled environments as a method to support player driven traversal design.

List of References

Adams, E. and Dormans, J. (2012) *Game mechanics: advanced game design*. New Riders.

Aramini, A.U., Lanzi, P.L. and Loiacono, D. (2018) 'An integrated framework for AI assisted level design in 2D platformers', in 2018 *IEEE Games, Entertainment, Media Conference (GEM)*, pp. 1–9. Available at: <https://doi.org/10.1109/GEM.2018.8516490>

Brühlmann, F. and Mekler, E.D. (2018) 'Surveys in games user research', in A. Drachen, P. Mirza-Babaei and L.E. Nacke (eds.), *Games user research*. Oxford University Press, pp. 141–162. Available at: <https://doi.org/10.1093/oso/9780198794844.003.0009>

Cardona-Rivera, R.E. and Young, R.M. (2014) 'A cognitivist theory of affordances for games', in *Proceedings of DiGRA 2013 Conference*. Digital Games Research Association. Available at: <https://doi.org/10.26503/dl.v2013i1.687>

Cerny Green, M., Khalifa, A., Barros, G.A.B., Nealen, A. and Togelius, J. (2018) 'Generating levels that teach mechanics', in *Proceedings of the 13th International Conference on the Foundations of Digital Games*. ACM. Available at: <https://doi.org/10.1145/3235765.3235820>

Compton, K. and Mateas, M. (2006) 'Procedural level design for platform games', *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 2(1), pp. 109–111. Available at: <https://doi.org/10.1609/aiide.v2i1.18755>

Dahlskog, S. and Togelius, J. (2012) 'Patterns and procedural content generation: revisiting Mario in World 1 Level 1', in *Proceedings of the First*

Workshop on Design Patterns in Games, pp. 1–8. ACM. Available at: <https://doi.org/10.1145/2427116.2427117>

Dormans, J. (2011) ‘Integrating emergence and progression’, in *Proceedings of DiGRA 2011 Conference: Think Design Play*. Digital Games Research Association. Available at: <https://doi.org/10.26503/dl.v2011i1.593>

Drachen, A. and Canossa, A. (2009) ‘Analyzing spatial user behavior in computer games using geographic information systems’, in *Proceedings of the 13th International MindTrek Conference*, pp. 182–189. ACM. Available at: <https://doi.org/10.1145/1621841.1621875>

Drachen, A., Canossa, A. and Sørensen, J.R.M. (2013) ‘Gameplay metrics in game user research: examples from the trenches’, in M. Seif El-Nasr, A. Drachen and A. Canossa (eds.), *Game analytics: maximizing the value of player data*. Springer, pp. 285–319. Available at: https://doi.org/10.1007/978-1-4471-4769-5_14

Gibson, J.J. (1979) *The ecological approach to visual perception*. Houghton Mifflin.

Hunicke, R., LeBlanc, M. and Zubek, R. (2004) ‘MDA: a formal approach to game design and game research’, in *Challenges in Game Artificial Intelligence: Papers from the 2004 AAAI Workshop*. AAAI Press, Technical Report WS-04-04. Available at: <https://aaai.org/papers/ws04-04-001-mda-a-formal-approach-to-game-design-and-game-research/> (Accessed: 21 August 2026).

IJsselsteijn, W.A., de Kort, Y.A.W. and Poels, K. (2013) *The Game Experience Questionnaire*. Technische Universiteit Eindhoven.

Juul, J. (2005) *Half-real: video games between real rules and fictional worlds*. MIT Press.

Linderoth, J. (2010) ‘Why gamers don’t learn more: an ecological approach to games as learning environments’, in *Proceedings of Nordic DiGRA 2010*

Conference: *Experiencing Games: Games, Play, and Players*. Digital Games Research Association. Available at: <https://doi.org/10.26503/dl.v2010i1.510>

Maddy Makes Games Inc. (2018) *Celeste* [Video game]. Maddy Makes Games Inc. Available at: <https://store.steampowered.com/app/504230/Celeste/> (Accessed: 21 August 2026).

Naftis, M., Tsatiris, G. and Karpouzis, K. (2021) 'How camera placement affects gameplay in video games', *arXiv*. Available at: <https://doi.org/10.48550/arXiv.2109.03750>

Nintendo (1981) *Donkey Kong* [Video game]. Nintendo. Available at: <https://www.nintendo.co.jp/corporate/en/history/index.html> (Accessed: 21 August 2026).

Nintendo (1996) *Super Mario 64* [Video game]. Nintendo. Available at: <https://mario.nintendo.com/history/> (Accessed: 21 August 2026).

Norman, D.A. (2013) *The design of everyday things*. Revised and expanded edn. Basic Books.

Pichlmair, M. and Johansen, M. (2022) '*Designing game feel: a survey*', *IEEE Transactions on Games*, 14(2), pp. 138–152. Available at: <https://doi.org/10.1109/TG.2021.3072241>

Salen, K. and Zimmerman, E. (2004) *Rules of play: game design fundamentals*. MIT Press.

Sarkar, A., Summerville, A., Snodgrass, S., Bentley, G. and Osborn, J. (2020) 'Exploring level blending across platformers via paths and affordances', *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 16(1), pp. 280–286. Available at: <https://doi.org/10.1609/aiide.v16i1.7442>

Smith, G., Cha, M. and Whitehead, J. (2008) 'A framework for analysis of 2D platformer levels', in *Proceedings of Sandbox '08: ACM SIGGRAPH*

Symposium on Video Games, pp. 75–80. ACM. Available at: <https://doi.org/10.1145/1401843.1401858>

Soler-Adillon, J. (2019) 'The open, the closed and the emergent: theorizing emergence for videogame studies', *Game Studies*, 19(2). Available at: <https://gamestudies.org/1902/articles/soleradillon> (Accessed: 21 August 2026).

Sweetser, P. and Wyeth, P. (2005) 'GameFlow: a model for evaluating player enjoyment in games', *Computers in Entertainment*, 3(3), p. 3. Available at: <https://doi.org/10.1145/1077246.1077253>

Wallner, G. (2013) 'Play-Graph: a methodology and visualization approach for the analysis of gameplay data', in *Proceedings of the International Conference on the Foundations of Digital Games*, pp. 253–260. Foundations of Digital Games.

Wallner, G. and Kriglstein, S. (2013) 'Visualization-based analysis of gameplay data: a review of literature', *Entertainment Computing*, 4(3), pp. 143–155. Available at: <https://doi.org/10.1016/j.entcom.2013.02.002>

Wardrip-Fruin, N., Mateas, M., Dow, S. and Sali, S. (2009) 'Agency reconsidered', in *Proceedings of DiGRA 2009 Conference: Breaking New Ground: Innovation in Games, Play, Practice and Theory*. Digital Games Research Association. Available at: <https://doi.org/10.26503/dl.v2009i1.369>

Wehbe, R.R., Mekler, E.D., Schaekermann, M., Lank, E. and Nacke, L.E. (2017) 'Testing incremental difficulty design in platformer games', in *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, pp. 5109–5113. ACM. Available at: <https://doi.org/10.1145/3025453.3025697>

Bibliography

Chang, A.Y. (2019) *Playing Nature: Ecology in Video Games*. University of Minnesota Press.

Appendices

Appendix A – Questionnaire

Consent: This study is being conducted as part of a university dissertation. Your participation is voluntary. You may stop at any time without providing a reason. No personally identifiable information will be collected, and all responses will remain anonymous. The data collected will be used solely for academic research purposes.

How often do you play games?

- Every day
- Once a week
- Once a month
- Very rarely

Note: Most of the questions will be in the form of a statement. Pick the option depending on if you agree or disagree.

"The dynamic level was more difficult than the static level." Would you agree or disagree?

- Strongly disagree
- Disagree
- Neutral
- Agree
- Strongly agree

"I understood what each cloud type could do in the dynamic level." Would you agree or disagree?

- Strongly disagree
- Disagree
- Neutral
- Agree
- Strongly agree

Please briefly explain how you figured out what the clouds could do, or what made it difficult to understand. (1–2 sentences)

"The clouds in the dynamic level felt like something I was actively interacting with rather than just an obstacle to get past." Would you agree or disagree?

- Strongly disagree
- Disagree
- Neutral
- Agree
- Strongly agree

"I felt free to approach the dynamic level in my own way rather than being restricted to one correct path." Would you agree or disagree?

- Strongly disagree
- Disagree
- Neutral
- Agree
- Strongly agree

Which cloud type did you use the most in dynamic level? (*select one*)

- Bouncy
- Moveable
- Breaking

"My approach to navigating the level changed between the static level and the dynamic level." Would you agree or disagree?

- Strongly disagree
- Disagree
- Neutral
- Agree
- Strongly agree

Briefly describe how your navigation approach differed between the two levels. (1–2 sentences)

"I found the dynamic level more enjoyable than the static level." Would you agree or disagree?

- Strongly disagree
- Disagree
- Neutral
- Agree
- Strongly agree

Briefly explain why. (1–2 sentences)

Any other thoughts on how the two levels felt different to play? (Optional)

Log Files

You can find the log files in the game folder.

[Extracted Game Folder] -> [ResearchProject] -> [PlaytestData] -> Log files

There will be two log files. One for the Static Level and one for Dynamic Level.

Submit them both please.

Note: If you have any problem with locating the files, please contact me.

Log Files Submission